

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

**Time Duration: 1 Hour
Total Marks:30**

Annexure A

Resolution Algorithm Implementation

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Resolution Algorithm Implementation

Question 1 – Rain and Wet Ground

Knowledge Base

1. If it rains, then the ground becomes wet.
2. It is raining.

Goal

Prove that **the ground is wet** using the Resolution Algorithm.

Tasks

1. Represent the statements in propositional logic.
2. Convert all statements into CNF.

Negate the goal.

1. Apply the Resolution Algorithm step by step.
2. Show the Empty Clause ($\square$) if the goal is proved.

Question 2 – Student Assignment Submission

Knowledge Base

1. If a student submits the assignment, then the student receives internal marks
2. Rahul submitted the assignment.

Goal

Prove that **Rahul receives internal marks** using the Resolution Algorithm.

Tasks

1. Write the propositional logic expressions.
2. Convert the premises into CNF.

Negate the goal.

1. Apply the Resolution Algorithm.
2. Conclude whether the goal is proved.

Question 3 – Library Membership

Knowledge Base

1. If a person is a library member, then the person can borrow books.
2. Priya is a library member.

Goal

1. Prove that **Priya can borrow books** using the Resolution Algorithm.

Tasks

1. Represent the knowledge base using propositional logic.
2. Convert each statement into CNF.

Negate the goal.

1. Perform resolution until no new clauses can be generated.
2. State the final conclusion.

Question 4 – Placement Eligibility

Knowledge Base

1. If a student clears the aptitude test, then the student is eligible for placement.
2. Arun cleared the aptitude test.

Goal

1. Prove that **Arun is eligible for placement** using the Resolution Algorithm.

Tasks

1. Convert the statements into propositional logic.
2. Write the CNF clauses.
3. Negate the goal.
4. Apply the Resolution Algorithm step by step.
5. Draw the resolution tree and conclude.

Question 5 – Access Control System

Knowledge Base

1. If a user enters the correct password, then the user is authenticated.
2. If a user is authenticated, then the user is granted access.
3. The user entered the correct password.

Goal

Prove that **the user is granted access** using the Resolution Algorithm.

Tasks

1. Represent the knowledge base in propositional logic.
2. Convert all implications into CNF.
3. Negate the goal.
4. Apply the Resolution Algorithm to derive the Empty Clause ($\square$).
5. Explain each resolution step and write the final conclusion.

1)

Resolution Algorithm Implementation: Rain and Wet Ground

1. Problem Statement

A knowledge base contains the rule “**If it rains, then the ground becomes wet**” and the fact “**It is raining.**” Using the **Resolution Algorithm**, prove that the ground is wet. The proof should include propositional logic representation, CNF conversion, negation of the goal, resolution steps, and the derivation of the **Empty Clause** ($\square$).

2. Propositional Logic Representation

Let:

- **R** = It is raining.
- **W** = The ground is wet.

Rule 1

“If it rains, then the ground becomes wet.”

$R \rightarrow W$

Fact

“It is raining.”

R

Goal

“The ground is wet.”

W

3. CNF Conversion

The rule is:

$R \rightarrow W$

Eliminate the implication:

$\neg R \vee W$

Therefore, the CNF clause is:

$\neg R \vee W$

The fact:

R

So the knowledge base in CNF is:

C1: $\neg R \vee W$

C2: R

4. Negate the Goal

The goal is:

W

For Resolution Refutation, negate the goal:

$\neg W$

Add it to the knowledge base.

Therefore:

C1: $\neg R \vee W$

C2: R

C3: $\neg W$

5. Resolution Algorithm

Resolution Step 1

Take:

C1: $\neg R \vee W$

and:

C2: R

Resolve on **R**.

The complementary literals are:

$\neg R$

and:

R

After resolution:

W

So we derive:

C4: W

Resolution Step 2

Now take:

C4:W

and the negated goal:

C3:¬W

Resolve on **W**.

The complementary literals are:

W

and:

¬W

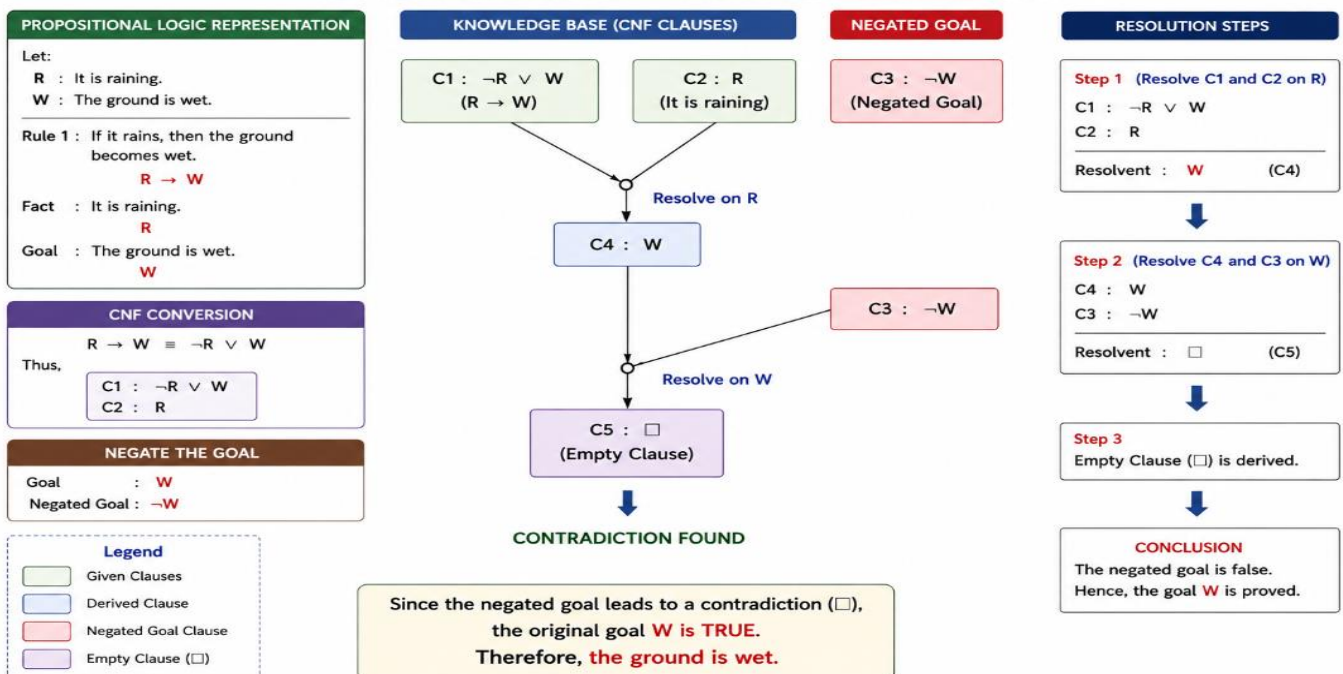
Therefore:

□

The symbol □ represents the **Empty Clause**.

6. Resolution Proof

RESOLUTION PROOF – RAIN AND WET GROUND



7. Python Code

class ResolutionAgent:

def __init__(self):

Knowledge Base

self.clauses = [

{"~R", "W"}, # R -> W

{"R"} # It is raining

]

Negated goal

self.goal = {"~W"}

self.derived_clauses = []

def display_knowledge_base(self):

print("\nKNOWLEDGE BASE")

print("=" * 50)

print("C1 : ~R OR W")

print("C2 : R")

print("\nGoal : W")

print("Negated Goal : ~W")

def resolve(self, clause1, clause2):

resolvents = []

for literal in clause1:

complementary = (

literal[1:]

if literal.startswith("~")

else "~" + literal

)

if complementary in clause2:

new_clause = (

clause1 - {literal}

) | (

clause2 - {complementary}

)

resolvents.append(new_clause)

return resolvents

def resolution_algorithm(self):

print("\nRESOLUTION ALGORITHM")

```

print("=" * 50)

clauses = list(self.clauses)

# Add negated goal
clauses.append(self.goal)

print("\nInitial Clauses:")

for index, clause in enumerate(clauses, 1):
    print(
        f"C{index} : "
        + " OR ".join(sorted(clause))
    )

step = 1

while True:

    new_clauses = []

    for i in range(len(clauses)):

        for j in range(i + 1, len(clauses)):

            resolvents = self.resolve(
                clauses[i],
                clauses[j]
            )

            for resolvent in resolvents:

                print(
                    f"\nResolution Step {step}"
                )

                print(
                    "Clause 1 :",
                    clauses[i]
                )

                print(
                    "Clause 2 :",
                    clauses[j]
                )

                print(
                    "Result  :",
                    resolvent
                )

            step += 1

```



```

        # Empty clause means contradiction
        if len(resolvent) == 0:

            print(
                "\nEmpty Clause ( $\square$ ) "
                "obtained."
            )

            print(
                "Goal W is PROVED."
            )

            return True

        if resolvent not in clauses:
            new_clauses.append(
                resolvent
            )

    if not new_clauses:

        print(
            "\nNo new clauses can be generated."
        )

        print(
            "Goal cannot be proved."
        )

        return False

    clauses.extend(new_clauses)

```

```

def main():

    agent = ResolutionAgent()

    agent.display_knowledge_base()

    result = agent.resolution_algorithm()

    print("\nFINAL CONCLUSION")
    print("=" * 50)

    if result:
        print(
            "The ground is wet because it is raining."
        )
    else:
        print(

```

```
"The ground being wet could not be proved."  
)
```

```
if __name__ == "__main__":  
    main()
```

8. Expected Output

```
FINAL CONCLUSION
```

```
=====
```

```
The ground is wet because it is raining.
```

```
Process finished with exit code 0
```

9. Report

The given knowledge base was successfully solved using the **Resolution Algorithm**. The rule $R \rightarrow W$ was converted into the CNF clause $\neg R \vee W$, while the fact that it is raining was represented as R . The goal W was negated to obtain $\neg W$. Resolution between $\neg R \vee W$ and R produced W . Resolving W with the negated goal $\neg W$ produced the **Empty Clause** ($\square$). Therefore, the goal “the ground is wet” is **successfully proved**.

Student Assignment Submission

1. Problem Statement

A knowledge base contains the rule “**If a student submits the assignment, then the student receives internal marks**” and the fact “**Rahul submitted the assignment.**” Using the **Resolution Algorithm**, prove that **Rahul receives internal marks**. The required steps are propositional logic representation, CNF conversion, negation of the goal, resolution, and final conclusion.

2. Propositional Logic Representation

Let:

- **S** = Rahul submitted the assignment.
- **M** = Rahul receives internal marks.

Rule

$S \rightarrow M$

Fact

S

Goal

M

Therefore:

$S \rightarrow M, S :: M$

3. CNF Conversion

Given:

$S \rightarrow M$

Eliminate implication:

$\neg S \vee M$

The fact is:

S

Therefore, the CNF knowledge base is:

C1 : $\neg S \vee M$

C2 : S

4. Negate the Goal

The goal is:

M

For Resolution Refutation, negate the goal:

$\neg M$

Add it to the knowledge base:

C1 : $\neg S \vee M$

C2 : S

C3 : $\neg M$

5. Resolution Algorithm

Resolution Step 1

Take:

C1: $\neg S \vee M$

and:

C2: S

Resolve on **S**.

The complementary literals are:

$\neg S$

and:

S

Therefore:

M

New clause:

C4 : M

Resolution Step 2

Now take:

C4: M

and the negated goal:

C3: $\neg M$

Resolve on **M**:

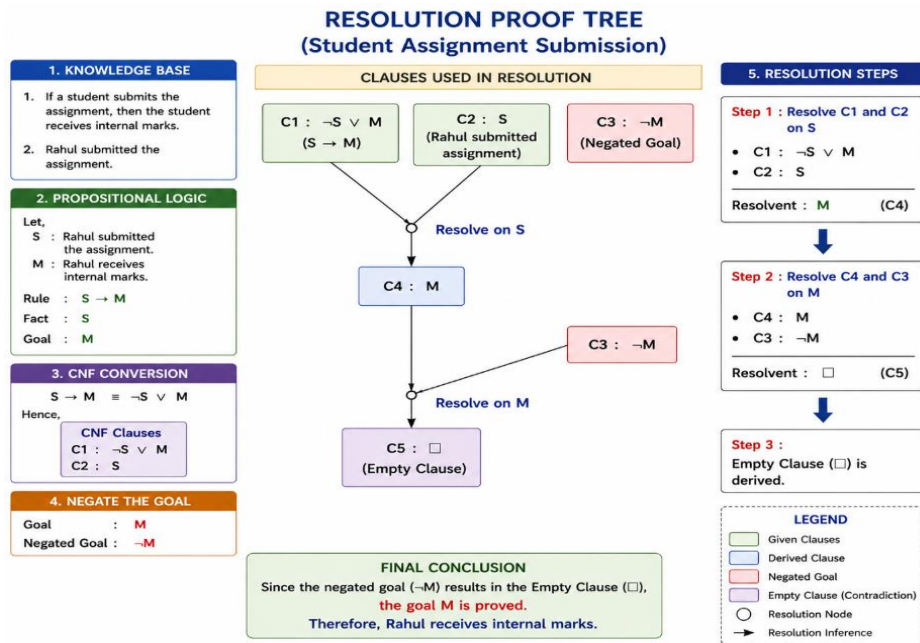
M, $\neg M$

Therefore:

□

The **Empty Clause** (□) is obtained.

6. Resolution Proof Tree



7. Final Conclusion

Since the negated goal $\neg M$ results in a contradiction and produces the **Empty Clause** (□):

M=True

Therefore:

Rahul receives internal marks.

8. Python Code

```
class ResolutionAgent:
```

```
    def __init__(self):
```

```
        # Knowledge Base
```

```
        self.clauses = [
```

```
            {"~S", "M"}, # If S then M
```

```
            {"S"}        # Rahul submitted assignment
```

```
        ]
```

```
        # Negated goal
```

```
        self.negated_goal = {"~M"}
```

```
    def display_knowledge_base(self):
```

```

print("\nKNOWLEDGE BASE")
print("=" * 50)

print("C1 : ~S OR M")
print("C2 : S")

print("\nGoal      : M")
print("Negated Goal : ~M")

def resolve(self, clause1, clause2):

    resolvents = []

    for literal in clause1:

        if literal.startswith("~"):
            complementary = literal[1:]
        else:
            complementary = "~" + literal

        if complementary in clause2:

            new_clause = (
                clause1 - {literal}
            ) | (
                clause2 - {complementary}
            )

            resolvents.append(new_clause)

    return resolvents

def resolution_algorithm(self):

    print("\nRESOLUTION ALGORITHM")
    print("=" * 50)

    clauses = list(self.clauses)

    # Add negated goal
    clauses.append(self.negated_goal)

    print("\nInitial Clauses:")

    for i, clause in enumerate(clauses, 1):

        print(
            f"C{i} : "
            + " OR ".join(sorted(clause))
        )

```

```

step = 1

while True:

    new_clauses = []

    for i in range(len(clauses)):

        for j in range(i + 1, len(clauses)):

            resolvents = self.resolve(
                clauses[i],
                clauses[j]
            )

            for resolvent in resolvents:

                print(
                    f"\nResolution Step {step}"
                )

                print(
                    "Clause 1 :",
                    clauses[i]
                )

                print(
                    "Clause 2 :",
                    clauses[j]
                )

                print(
                    "Resolvent :",
                    resolvent
                )

                step += 1

            # Empty clause
            if len(resolvent) == 0:

                print(
                    "\nEmpty Clause ( $\square$ ) "
                    "obtained."
                )

                print(
                    "Goal M is PROVED."
                )

            return True

```

```

        if resolvent not in clauses:

            new_clauses.append(
                resolvent
            )

    if not new_clauses:

        print(
            "\nNo new clauses can be generated."
        )

        return False

    clauses.extend(new_clauses)

def final_result(self, result):

    print("\nFINAL CONCLUSION")
    print("=" * 50)

    if result:

        print(
            "Rahul receives internal marks."
        )

    else:

        print(
            "Rahul receiving internal marks "
            "could not be proved."
        )

def main():

    agent = ResolutionAgent()

    agent.display_knowledge_base()

    result = agent.resolution_algorithm()

    agent.final_result(result)

if __name__ == "__main__":
    main()

```

9. Output


```
Empty Clause ( $\square$ ) obtained.
```

```
Goal M is PROVED.
```

```
FINAL CONCLUSION
```

```
=====
```

```
Rahul receives internal marks.
```

```
Process finished with exit code 0
```

10. Report

The **Resolution Algorithm** was successfully applied to prove that Rahul receives internal marks. The implication $S \rightarrow M$ was converted into the CNF clause $\neg S \vee M$. The fact S and the negated goal $\neg M$ were then added to the knowledge base. Resolving $\neg S \vee M$ with S produced M , and resolving M with $\neg M$ produced the **Empty Clause** ($\square$). Therefore, the goal is proved and **Rahul receives internal marks**.

Library Membership

1. PROBLEM STATEMENT

A knowledge base contains the rule “**If a person is a library member, then the person can borrow books**” and the fact “**Priya is a library member.**” Using the **Resolution Algorithm**, prove that **Priya can borrow books**. The solution should represent the knowledge base using propositional logic, convert the statements into CNF, negate the goal, perform resolution, and state the final conclusion.

2. PROPOSITIONAL LOGIC REPRESENTATION

Let:

- **L** = Priya is a library member.
- **B** = Priya can borrow books.

Rule

If a person is a library member, then the person can borrow books.

$L \rightarrow B$

Fact

Priya is a library member.

L

Goal

Priya can borrow books.

B

Therefore:

$L \rightarrow B, L \therefore B$

3. CNF CONVERSION

The rule is:

$L \rightarrow B$

Eliminate the implication:

$\neg L \vee B$

The given fact is:

L

Therefore, the CNF clauses are:

$C1 : \neg L \vee B$

$C2 : L$

4. NEGATION OF THE GOAL

The goal is:

B

For Resolution Refutation, negate the goal:

$\neg B$

Add the negated goal to the knowledge base.

Therefore:

$C1 : \neg L \vee B$

$C2 : L$

$C3 : \neg B$

5. RESOLUTION ALGORITHM

Resolution Step 1

Take:

$C1: \neg L \vee B$

and:

$C2:L$

The complementary literals are:

$\neg L$

and:

L

Resolving them gives:

B

New clause:

$C4 : B$

Resolution Step 2

Now take:

C4:B

and:

C3:¬B

The complementary literals are:

B

and:

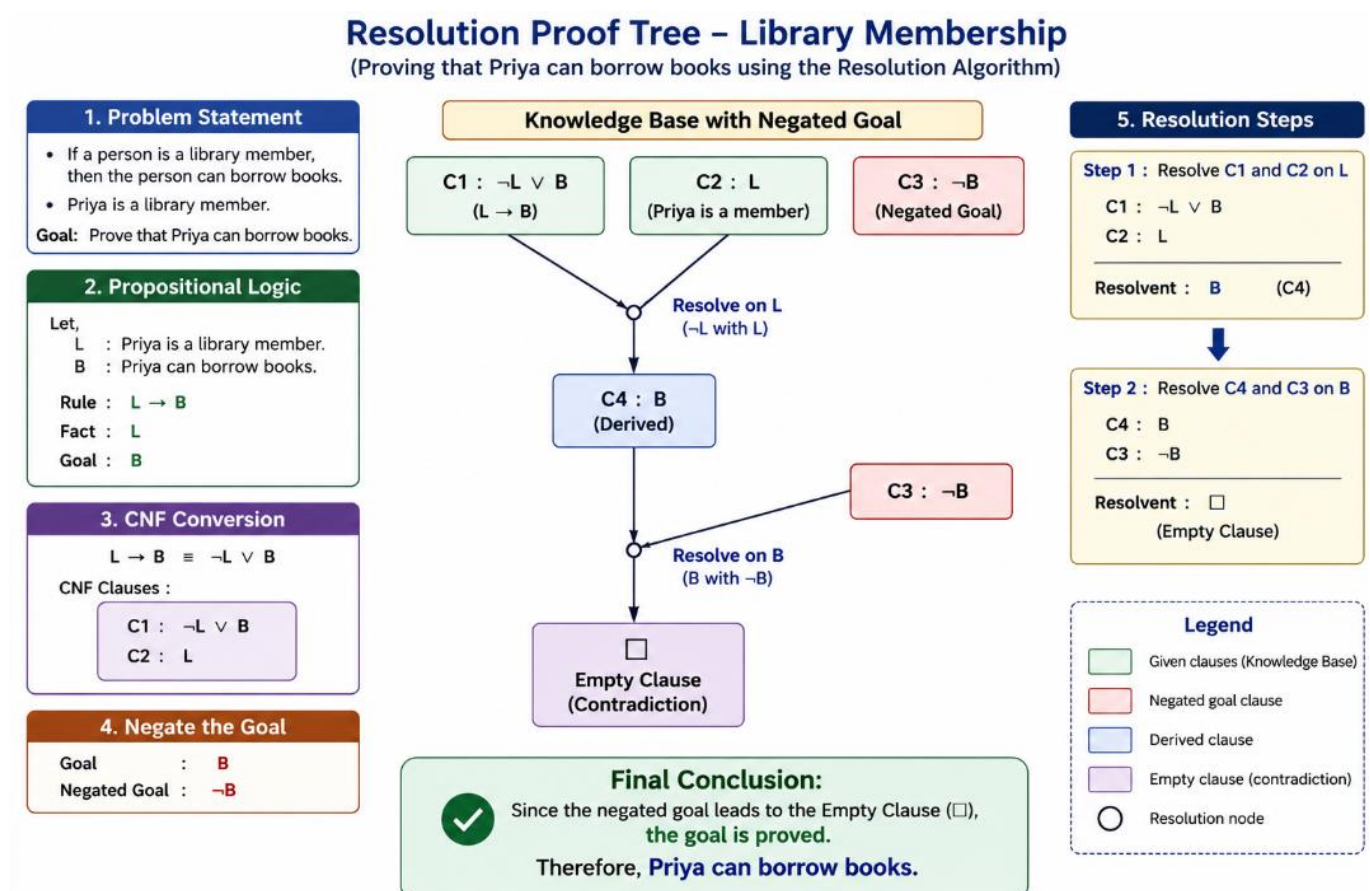
¬B

Resolving them gives:

□

The **Empty Clause** (□) is obtained.

6. RESOLUTION PROOF TREE



7. FINAL CONCLUSION

Since the negated goal:

$\neg B$

leads to the **Empty Clause** ($\square$), the negated goal is contradictory.

Therefore:

$B = \text{True}$

Hence:

Priya can borrow books.

8. PYTHON CODE

```
class ResolutionAgent:
```

```
    def __init__(self):
```

```
        # Knowledge Base
```

```
        self.clauses = [
```

```
            {"~L", "B"},    # L -> B
```

```
            {"L"}          # Priya is a library member
```

```
        ]
```

```
        # Negated goal
```

```
        self.negated_goal = {"~B"}
```

```
    def display_knowledge_base(self):
```

```
        print("\nKNOWLEDGE BASE")
```

```
        print("=" * 50)
```

```
        print("C1 : ~L OR B")
```

```
        print("C2 : L")
```

```
        print("\nGoal      : B")
```

```
        print("Negated Goal : ~B")
```

```
    def resolve(self, clause1, clause2):
```

```
        resolvents = []
```

```
        for literal in clause1:
```

```
            if literal.startswith("~"):
```

```
                complementary = literal[1:]
```

```
            else:
```

```
                complementary = "~" + literal
```

```
            if complementary in clause2:
```

```

        new_clause = (
            clause1 - {literal}
        ) | (
            clause2 - {complementary}
        )

        resolvents.append(new_clause)

    return resolvents

def resolution_algorithm(self):

    print("\nRESOLUTION ALGORITHM")
    print("=" * 50)

    clauses = list(self.clauses)

    # Add the negated goal
    clauses.append(self.negated_goal)

    print("\nInitial Clauses:")

    for index, clause in enumerate(clauses, 1):

        print(
            f"C{index} : "
            + " OR ".join(sorted(clause))
        )

    step = 1

    while True:

        new_clauses = []

        for i in range(len(clauses)):

            for j in range(i + 1, len(clauses)):

                resolvents = self.resolve(
                    clauses[i],
                    clauses[j]
                )

                for resolvent in resolvents:

                    print(
                        f"\nResolution Step {step}"
                    )

                    print(
                        "Clause 1 :",

```

```

        clauses[i]
    )

    print(
        "Clause 2 :",
        clauses[j]
    )

    print(
        "Resolvent :",
        resolvent
    )

    step += 1

    # Empty clause means contradiction
    if len(resolvent) == 0:

        print(
            "\nEmpty Clause ( $\square$ ) "
            "obtained."
        )

        print(
            "Goal B is PROVED."
        )

        return True

    if resolvent not in clauses:

        new_clauses.append(
            resolvent
        )

    if not new_clauses:

        print(
            "\nNo new clauses can be generated."
        )

        return False

    clauses.extend(new_clauses)

def display_result(self, result):

    print("\nFINAL CONCLUSION")
    print("=" * 50)

    if result:

```

```

        print(
            "Priya can borrow books."
        )

    else:

        print(
            "Priya can borrow books "
            "could not be proved."
        )

def main():

    agent = ResolutionAgent()

    agent.display_knowledge_base()

    result = agent.resolution_algorithm()

    agent.display_result(result)

if __name__ == "__main__":
    main()

```

9. EXPECTED OUTPUT

```

Empty Clause (□) obtained.
Goal B is PROVED.

FINAL CONCLUSION
=====
Priya can borrow books.

Process finished with exit code 0

```

10. REPORT

The **Resolution Algorithm** was successfully applied to prove that Priya can borrow books. The rule $L \rightarrow B$ was converted into the CNF clause $\neg L \vee B$. The fact L and the negated goal $\neg B$ were then added to the knowledge base. Resolving $\neg L \vee B$ with L produced B. Resolving B with the negated goal $\neg B$ produced the **Empty Clause** ($\square$). Therefore, the goal is proved and **Priya can borrow books**.

4)

Placement Eligibility

1. PROBLEM STATEMENT

A college placement system contains a logical rule stating that **if a student clears the aptitude test, then the student is eligible for placement**. It is also known that **Arun cleared the aptitude test**. Using the **Resolution Algorithm**, prove that **Arun is eligible for placement**.

2. PROPOSITIONAL LOGIC

Let:

- **A** = Arun cleared the aptitude test.
- **E** = Arun is eligible for placement.

Rule

$A \rightarrow E$

Fact

A

Goal

E

Therefore:

$A \rightarrow E, A \therefore E$

3. CNF CONVERSION

Given:

$A \rightarrow E$

Eliminate the implication:

$\neg A \vee E$

The given fact is:

A

Therefore, the CNF clauses are:

C1 : $\neg A \vee E$

C2 : A

4. NEGATE THE GOAL

The goal is:

E

Negate the goal for Resolution Refutation:

$\neg E$

Therefore, the complete set of clauses is:

C1 : $\neg A \vee E$

C2 : A

C3 : $\neg E$

5. RESOLUTION ALGORITHM

Resolution Step 1

Take:

C1: $\neg A \vee E$

and:

C2: A

The complementary literals are:

$\neg A$

and:

A

Resolving on **A**:

$(\neg A \vee E), A$

gives:

E

New clause:

C4 : E

Resolution Step 2

Now resolve:

C4:E

with:

C3: $\neg E$

The complementary literals are:

E

and:

$\neg E$

Therefore:

□

The **Empty Clause** (□) is obtained.

6. RESOLUTION PROOF TREE

C1 : $\neg A \vee E$

+

C2 : A

|

Resolve on A

|

↓

C4 : E

+

C3 : $\neg E$

|

Resolve on E

|

↓

□

Empty Clause

Resolution Tree Explanation

C1 + C2

↓

E

↓

E + $\neg E$

↓

□

The Empty Clause represents a contradiction.

7. FINAL CONCLUSION

The negated goal:

$\neg E$

produces a contradiction with the derived clause:

E

and results in the Empty Clause:

□

Therefore:

E=True

Final Answer

Arun is eligible for placement.

8. PYTHON CODE

class ResolutionAgent:

def __init__(self):

 # Knowledge Base

 self.clauses = [

 {"~A", "E"}, # A -> E

 {"A"} # Arun cleared aptitude test

]

 # Negated goal

 self.negated_goal = {"~E"}

def display_knowledge_base(self):

 print("\nKNOWLEDGE BASE")

 print("=" * 50)

 print("C1 : ~A OR E")

 print("C2 : A")

 print("\nGoal : E")

 print("Negated Goal : ~E")

def resolve(self, clause1, clause2):

 resolvents = []

 for literal in clause1:

 if literal.startswith("~"):

```

        complementary = literal[1:]
    else:
        complementary = "~" + literal

    if complementary in clause2:

        new_clause = (
            clause1 - {literal}
        ) | (
            clause2 - {complementary}
        )

        resolvents.append(new_clause)

    return resolvents

def resolution_algorithm(self):

    print("\nRESOLUTION ALGORITHM")
    print("=" * 50)

    clauses = list(self.clauses)

    # Add negated goal
    clauses.append(self.negated_goal)

    print("\nInitial Clauses:")

    for index, clause in enumerate(clauses, 1):

        print(
            f"C{index} : "
            + " OR ".join(sorted(clause))
        )

    step = 1

    while True:

        new_clauses = []

        for i in range(len(clauses)):

            for j in range(i + 1, len(clauses)):

                resolvents = self.resolve(
                    clauses[i],
                    clauses[j]
                )

                for resolvent in resolvents:

```

```
print(
    f"\nResolution Step {step}"
)
```

```
print(
    "Clause 1 :",
    clauses[i]
)
```

```
print(
    "Clause 2 :",
    clauses[j]
)
```

```
print(
    "Resolvent :",
    resolvent
)
```

```
step += 1
```

```
# Empty clause
if len(resolvent) == 0:
```

```
    print(
        "\nEmpty Clause ( $\square$ ) "
        "obtained."
    )
```

```
    print(
        "Goal E is PROVED."
    )
```

```
    return True
```

```
if resolvent not in clauses:
```

```
    new_clauses.append(
        resolvent
    )
```

```
if not new_clauses:
```

```
    print(
        "\nNo new clauses can be generated."
    )
```

```
    return False
```

```
clauses.extend(new_clauses)
```

```
def display_result(self, result):
```

```

print("\nFINAL CONCLUSION")
print("=" * 50)

if result:

    print(
        "Arun is eligible for placement."
    )

else:

    print(
        "Arun's placement eligibility "
        "could not be proved."
    )

def main():

    agent = ResolutionAgent()

    agent.display_knowledge_base()

    result = agent.resolution_algorithm()

    agent.display_result(result)

if __name__ == "__main__":
    main()

```

9.EXPECTED OUTPUT

```
FINAL CONCLUSION
```

```
=====
```

```
Arun is eligible for placement.
```

```
Process finished with exit code 0
```

10. REPORT

The Placement Eligibility problem was successfully solved using the **Resolution Algorithm**. The implication $A \rightarrow E$ was converted into the CNF clause $\neg A \vee E$. The fact A and the negated goal $\neg E$ were then included in the knowledge base. Resolving $\neg A \vee E$ with A produced E . Resolving E with $\neg E$ produced the **Empty Clause** ($\square$).

Therefore, the negated goal leads to a contradiction and the original goal is proved. Hence, **Arun is eligible for placement**.

5)

Access Control System

1. PROBLEM STATEMENT

An access control system uses logical rules to determine whether a user should be granted access. The knowledge base contains three statements:

1. If a user enters the correct password, the user is authenticated.
2. If a user is authenticated, the user is granted access.
3. The user entered the correct password.

Using the **Resolution Algorithm**, prove that the **user is granted access**.

2. PROPOSITIONAL LOGIC REPRESENTATION

Let:

- **P** = User entered the correct password.
- **A** = User is authenticated.
- **G** = User is granted access.

Rule 1

$P \rightarrow A$

Rule 2

$A \rightarrow G$

Fact

P

Goal

G

Therefore:

$P \rightarrow A, A \rightarrow G, P \therefore G$

3. CNF CONVERSION

Rule 1

$P \rightarrow A$

Eliminate implication:

$\neg P \vee A$

Therefore:

C1 : $\neg P \vee A$

Rule 2

$A \rightarrow G$

Eliminate implication:

$\neg A \vee G$

Therefore:

C2 : $\neg A \vee G$

Fact

The user entered the correct password:

P

Therefore:

C3 : P

Complete CNF Knowledge Base

C1 : $\neg P \vee A$

C2 : $\neg A \vee G$

C3 : P

4. NEGATE THE GOAL

The goal is:

G

For Resolution Refutation, negate the goal:

$\neg G$

Add it to the knowledge base:

C1 : $\neg P \vee A$

C2 : $\neg A \vee G$

C3 : P

C4 : $\neg G$

5. RESOLUTION ALGORITHM

Resolution Step 1

Take:

C1: $\neg P \vee A$

and:

C3: P

The complementary literals are:

$\neg P$

and:

P

Resolve on **P**:

$(\neg P \vee A), P$

Therefore:

A

New clause:

C5 : A

This proves that the user is authenticated.

Resolution Step 2

Now take:

C2: $\neg A \vee G$

and:

C5: A

The complementary literals are:

$\neg A$

and:

A

Resolve on **A**:

$(\neg A \vee G), A$

Therefore:

G

New clause:

C6 : G

This proves that the user is granted access.

Resolution Step 3

Now take:

C6:G

and the negated goal:

C4:¬G

The complementary literals are:

G

and:

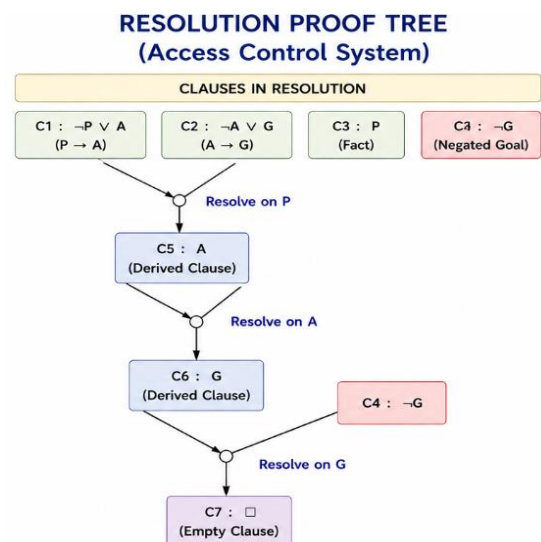
¬G

Therefore:

□

The **Empty Clause** (□) is obtained.

6.RESOLUTION PROOF TREE



7. FINAL CONCLUSION

The Resolution Algorithm derives:

$P \rightarrow A$

then:

$A \rightarrow G$

and finally:

G

Resolving the derived goal G with the negated goal $\neg G$ produces the **Empty Clause** ($\square$).

Therefore:

$G = \text{True}$

8. PYTHON CODE

class ResolutionAgent:

```
def __init__(self):
```

```
    # Knowledge Base
```

```
    self.clauses = [
```

```
        {"~P", "A"},    # P -> A
```

```
        {"~A", "G"},    # A -> G
```

```
        {"P"}           # Correct password entered
```

```
    ]
```

```
    # Negated goal
```

```
    self.negated_goal = {"~G"}
```

```
def display_knowledge_base(self):
```

```
    print("\nKNOWLEDGE BASE")
```

```
    print("=" * 55)
```

```
    print("C1 : ~P OR A")
```

```
    print("C2 : ~A OR G")
```

```
    print("C3 : P")
```

```
    print("\nGoal      : G")
```

```
    print("Negated Goal : ~G")
```

```
def resolve(self, clause1, clause2):
```

```
    resolvents = []
```

```
    for literal in clause1:
```

```
        if literal.startswith("~"):
```

```
            complementary = literal[1:]
```

```
        else:
```

```
            complementary = "~" + literal
```

```
    if complementary in clause2:
```

```

        new_clause = (
            clause1 - {literal}
        ) | (
            clause2 - {complementary}
        )

        resolvents.append(new_clause)

    return resolvents

def display_clause(self, clause):

    if len(clause) == 0:
        return "□"

    return " OR ".join(
        sorted(clause)
    )

def resolution_algorithm(self):

    print("\nRESOLUTION ALGORITHM")
    print("=" * 55)

    clauses = list(self.clauses)

    # Add the negated goal
    clauses.append(
        self.negated_goal
    )

    print("\nInitial Clauses:")

    for index, clause in enumerate(
        clauses,
        start=1
    ):

        print(
            f"C{index} : "
            + self.display_clause(clause)
        )

    step = 1

    while True:

        new_clauses = []

        for i in range(
            len(clauses)

```

```

):

for j in range(
    i + 1,
    len(clauses)
):

    resolvents = self.resolve(
        clauses[i],
        clauses[j]
    )

    for resolvent in resolvents:

        print(
            f"\nResolution Step {step}"
        )

        print(
            "Clause 1 :",
            self.display_clause(
                clauses[i]
            )
        )

        print(
            "Clause 2 :",
            self.display_clause(
                clauses[j]
            )
        )

        print(
            "Resolvent :",
            self.display_clause(
                resolvent
            )
        )

        step += 1

    # Empty clause
    if len(resolvent) == 0:

        print(
            "\nEmpty Clause ( $\square$ ) "
            "obtained."
        )

        print(
            "Goal G is PROVED."
        )

```

```

        return True

    if (
        resolvent not in clauses
        and
        resolvent not in new_clauses
    ):

        new_clauses.append(
            resolvent
        )

    if not new_clauses:

        print(
            "\nNo new clauses can be generated."
        )

        print(
            "Goal cannot be proved."
        )

        return False

    clauses.extend(
        new_clauses
    )

def display_result(self, result):

    print("\nFINAL CONCLUSION")
    print("=" * 55)

    if result:

        print(
            "The user is granted access."
        )

    else:

        print(
            "The user is NOT proved "
            "to have access."
        )

def main():

    agent = ResolutionAgent()

```



```
agent.display_knowledge_base()

result = agent.resolution_algorithm()

agent.display_result(result)

if __name__ == "__main__":
    main()
```

9.EXPECTED OUTPUT

```
FINAL CONCLUSION
=====
The user is granted access.

Process finished with exit code 0
```

10. REPORT

The Access Control System was successfully solved using the **Resolution Algorithm**. The two implications were converted into CNF as $\neg P \vee A$ and $\neg A \vee G$, while the fact P represented the correct password being entered. The goal G was negated to obtain $\neg G$.

First, resolving $\neg P \vee A$ with P produced A , proving authentication. Next, resolving $\neg A \vee G$ with A produced G , proving that access is granted. Finally, resolving G with the negated goal $\neg G$ produced the **Empty Clause** ($\square$). Therefore, the goal is proved: **the user is granted access**.